

اول از همه که واضحه برای اجرای این تمرین نیاز به pygame هست. بعد از نصب pygame برای اجرای برنامه فایل main را باز کرده و اجرا کنید.

خب از همین main.py شروع کنیم. برای اکثر متغیرها اینکه برای چه کاری تعریف شدند از روی اسمشون معلومه، مثلاً `choose_level_x` و `choose_level_y` برای تعیین مختصات شروع صفحه‌ی انتخاب سطح بازی هستند و `choose_level_width` و `choose_level_height` هم عرض و ارتفاع صفحه‌ی انتخاب سطح رو مشخص میکنن و یک متغیر از نوع `boolean` وجود داره که نشون میده الان باید این صفحه نشون داده بشه یا نه و خب مثلاً برای دکمه‌هایی که مربوط به صفحه‌ی انتخاب سطح میشن دیگه جداگانه این چیزا حساب نشده فقط یه دونه `pygame.Rect()` هست که باتوجه به اندازه‌های پنجره‌ای که بهش مربوط میشن اندازه هارو مشخص میکنه. `fps` اولیه 60 هست ولی خب باتوجه به سطحی که کاربر انتخاب میکنه تغییر میکنه و میتونه 100 و 150 هم باشه. درحلقه‌ی `running` اول که چک میکنه اگه طرف کاری کرده باشه که پنجره باید بسته میشد پنجره رو میبندد مثل دکمه ضربدر یا `Alt+F4`. بعد میاد چک میکنه اگه بازی درحال اجرا بود و `boolean` های مربوط به صفحه‌ی انتخاب سطح و صفحه‌ی پایان بازی (`game over`) مقدار `False` داشتن میاد و از تابع `game` که داخل `snake.py` هست متغیرهای `x` (مشخص کننده‌ی اینکه از لحاظ افقی سر مار داخل کدوم قسمت هست) `y` (همون کار `x` منتها از لحاظ عمودی) `food_x` و `food_y` که همون `x` و `y` هستند ولی برای محلی که غذا باید قرار بگیره، `endgame` که اگه سر مار به دیوارها یا خود بدن مار خورده باشه `True` میشه، `tails` که لیستی از اشیاء از کلاس `Tail` هست (حالا کلاس `Tail` چیه ← به غیر از سر مار که مختصاتش جداس بقیه دایره‌های مار هر کدوم که یک شیء از کلاس `Tail` هستند که مختصاتی که الان دارند و مختصاتی که قبل از مختصات فعلی داشتند برای استفاده برای دایره‌ای که بهشون وصل میشه، داخل یک لیست هم حرکات قبلی به مقدار کافی ذخیره میشه و یک رنگ تصادفی هم براش انتخاب میشه تا هر تیکه از دم یه رنگ داشته باشه که البته استفاده از این ویژگی فعلاً کامنت شده و هر بار یه رنگ تصادفی برای هرتیکه انتخاب میشه) میتونین به جای خط 129 فایل `main.py` خط 130 رو کامنت کنید تا رنگ هر تیکه ثابت باشه)) یک ایموجی هم میگیره برای تصویری که برای غذا میخواد نشون بده و یه متغیر هم که حرکتی که سر مار باید بکنه رو معلوم میکنه.

حالا خود تابع `game` که داخل `snake.py` هست ← داخل این تابع اول بررسی میشه که کاربر چه دکمه‌ای رو زده مثلاً اگه بالا رو زده باشه متغیر `go_up` مقدار `True` میگیره و متغیرهایی که مربوط به بقیه جهت‌ها هستند `False` میشن، البته قبلش جهت حرکت قبلی ذخیره میشه و اینکه مار روی خودش برنگرده هم کنترل میشه. یک متغیر به اسم `move_done` هست که مقدار `False` میگیره چون درادامه بررسی باید بررسی بشه که سر مار قبل از تغییر جهت حتماً داخل یه مربع باشه و دایره‌ی سر مار برخوردی با اضلاع مربع‌های کوچیک نداشته باشه و بعد تغییر جهت اتفاق بیفته یه چک هم میکنه اگه مختصات از صفحه رد شده `end_game` رو `True` میکنه و از تابع `game over` میاد بیرون و صفحه‌ی `game over` میاد ولی اگه برخورد نداشت میره داخل شروطی که برای هرجهت حرکت هست. از اونجایی که مثل هم هستن هرچهار شرط یکیشو میگم بقیش مثل همونه فقط جهتش فرق داره. خب برای `go_up` مثلاً، اول بررسی میشه اگه مختصات `x` بر `step` که همون اندازه ضلع مربع‌ها هست بخش پذیر بود اگه تغییر جهت لازم بود که اتفاق میفته اگه نه هم با توجه به همون جهت قبلی `y` آپدیت میشه. مثلاً اینجا که `go_up` هست مقدار `y` به اندازه‌ی `move_step` که ربطی به `step` نداره و یه عدد نسبتاً کوچیکی هست که باعث میشه حرکت مار روان باشه، کم بشه و درنتیجه سر مار میاد بالا. برای حرکت قبلی هم بررسی میشه که اگه سر مار دقیقاً داخل یکی از مربع‌های صفحه هست مختصات فعلیش برای دم‌هایی که قراره بهش اضافه بشن ذخیره بشه. اگر که مختصات مختصات سر مار و مختصات

غذا باهم تداخلی داشتن صدای خوردن غذا با تابع `eat_sound()` که داخل `sound.py` هست پخش میشه (داخل این تابع میتوان با تغییر عددی که داخل پرانتز `set_volume()` بین صفر و یک اندازه صدایی که پخش میشه رو کنترل کرد) بعد مختصات جدید غذا و لیست آپدیت شده‌ی دم ها و ایموجی جدید برای غذا توسط تابع `food_ate()` تعیین میشه (`food_ate()` ← اول مختصات جدید رو از تابع `food_coordinate()` میگیره که این تابع توضیح خاصی نداره صرفاً میاد به مختصات رندوم که داخل مربع ها باشه رو تولید میکنه. بعد میاد نگاه میکنه اگه قبلاً دمی داشته باشیم میاد مختصات قبلی آخرین دم رو میگیره و به عنوان مختصات فعلی دمی که قراره اضافه بشه میزاره و بعد از این شیء جدید `Tail` رو ساخت به لیست `tails` اضافه میکنه اگه هم که اولین دم هست میاد مختصات قبلی سر مار که ذخیره شده رو میذاره به جا مختصات فعلی دم و بعد اضافه میکنه به لیست `tails`) خب ادامه‌ی تابع `game`، داخل شروط جهت بودیم و داشتیم راجع به `go_up` میگفتیم. خب حالا اگه سر مار هنوز کامل داخل یک مربع نبود چی؟ خب اینجا میاد اول نگاه میکنه حرکت قبلی چی بوده و اونو انجام میده مثلاً اگه راست بوده میره راست و انقدر ادامه میده تا بالاخره وارد یکی از مربع ها بشه و بعد تغییر جهت رو انجام بده و بعد از تغییر جهت `move_done` مقدار `True` رو میگیره. بعد از این شروط هم میره برای ذخیره‌ی حرکت به این صورت که همیشه به مقدار خاصی که خودمون بهش میدیم حرکات قبلی رو نگه میداره مثلاً 8 حرکت قبلی دلایلش هم اینه که دمی که قراره کشیده بشه با یک فاصله‌ی مناسبی از دم جلویی خودش کشیده بشه (برای درک بهتر میتونید عدد `tail_distance` در خط 155 فایل `snake.py` رو عوض کنید اگه خیلی زیاد بشه فاصله بین `tail` ها هم زیاد میشه و سر مار میتونه از بین این فاصله رد شه! و اگه کم بشه هم دیگه خیلی `tail` ها میرن توهم تاجایی که بعد خوردن اولین غذا بازی به علت برخورد مار با خودش تموم میشه!)

بعد از گرفتن این مقادیر هم چک میکنه اگه نیازی به آپدیت کردن اهنگ پس زمینه بود مثل وقتی که به آهنگ تموم میشه اون رو آپدیت میکنه. بعد از اینها شروع میکنه به کشیدن صفحه اول که پس زمینه رو سیاه میکنه بعد برای نشون دادن امتیاز میاد رقم های امتیاز رو جدا میکنه و با ایموجی مربوط بهش داخل مختصات مشخصی قرار میده. بیشترین امتیاز هم سمت راست صفحه میذاره. اینجا به تیکه داریم از خط 117 تا 120 که کامنت شده که اگه کامنتش رو بردارین صفحه‌ی بازی خط‌کشی شده نشون داده میشه (برای اینکه ببینم وقتی میخواد مار مسیر رو عوض کنه حتماً داخل مربع ها اینکار رو انجام بده و خطوط رو قطع نکنه این خطارو گذاشتم) بعد سر مار رو که مشخصاتشو از قبل گرفته بود در اون جایی که باید باشه میکشه (راجع به `step` هم بگم که اندازه ضلع مربع های کوچیکی هست که اگه خط هارو بذارین دیده میشه) و بعد میره داخل لیست `tails` و هر `tail` مطابق با مختصاتی که داره میکشه و همینجا هم با `math.hypot` میاد بررسی میکنه که اگه فاصله سر مار از یکی دم ها کمتر از به اندازه ای بود بازی تموم شه.

بعد داخل مختصات غذا ایموجی غذا رو میذاره (اینکه ایموجی رو خود دم مار نیفته کنترل نشده) بقیش هم چیز خاصی نداره میاد بررسی میکنه اگه باید صفحه انتخاب سطح رو نشون بده میاد با توجه به همون متغیرهای که از قبل تعیین کردیم صفحه و دکمه هاش رو میکشه و هرکدوم از دکمه ها که کلیک شد به سری کارا رو انجام میده اگر هم که باید صفحه پایان بازی رو نشون بده که نشون میده با دکمه هاش

راجع به فایل های `score.py` و `sound.py` بگم چیز خاصی نداره که توضیح بخواد، یکی مسئول آپدیت امتیاز بعد اینکه غذا خورده شد هست اینجوری که بعد هربار که غذا رو میخوره مار تابع `add_score()` به امتیاز اضافه میکنه و کارای مربوط به ذخیره ی بیشترین امتیاز داخل دیتابیس (اینجوری گذاشتم

که حتی وقتی برنامه بسته شد هم نگه میداره بیشترین امتیاز رو و بعد اینکه دوباره اجرا بشه بیشترین امتیاز مقدار داره) اون یکی هم کارای مربوط به پخش
صدا رو انجام میده. راجع به تابع `reset_snake()` که داخل `snake.py` هست هم بگم که اگه کاربر دکمه بازی دوباره رو زد متغیرهایی که لازم باشه رو
ریست میکنه

اون `build` و `dist` و `main.spec` هم برای آپ ساختن به وجود امدند که متاسفانه با شکست سنگینی روبرو شد!

اگه غلط املایی زیاد بود دیگه به بزرگواری خود ببخشید 🙏🏻

همین دیگه با تشکر سید علیرضا حسینی 403521231